

ClaimSense AI

Project Interview Preparation Handbook

A beginner-friendly, interview-focused study guide built from the provided project files.

Prepared for technical interview revision. Generated from the local repository at
C:/Users/LENOVO/Downloads/ClaimSenseAi-main/ClaimSenseAi-main.

Table of Contents

- Part 1 - Project At A Glance
- Part 2 - Explain The Project Like I Am A Beginner
- Part 3 - System Architecture
- Part 4 - Complete End-To-End Execution
- Part 5 - Technologies Used
- Part 6 - Core Concepts I Must Understand
- Part 7 - Code Walkthrough
- Part 8 - Data Flow
- Part 9 - Database
- Part 10 - AI / Machine Learning Explanation
- Part 11 - API Explanation
- Part 12 - Why This Technology / Design?
- Part 13 - Challenges And Solutions
- Part 14 - Limitations
- Part 15 - Scalability
- Part 16 - Security
- Part 17 - Testing And Debugging
- Part 18 - Deployment
- Part 19 - Interview Question Bank
- Part 20 - Cross-Questioning Simulation
- Part 21 - HR + Project Questions
- Part 22 - 2-Minute Presentation
- Part 23 - Rapid Revision Sheet
- Part 24 - Flashcards
- Part 25 - Final Checklist

Accuracy Notes

This handbook separates project facts from interpretations. A project fact means it is visible in the provided code or documentation. An interpretation means it is a reasonable way to explain or extend the project in an interview, but it is not fully implemented in the provided files.

Important documentation/code mismatch: `architecture_overview.md` describes semantic search and mentions Gemini 2.0 Flash for ingest plus Gemini 3.0 Flash Preview for retrieval. The actual Supabase functions use `google/gemini-2.5-flash` through `https://ai.gateway.lovable.dev`. The actual retrieval ranking in both the browser fallback and Edge Function is keyword/token scoring, not a true embedding/vector search.

Remember This: In an interview, say the current project behaves like a lightweight RAG assistant. It has retrieval plus grounded generation, but it does not currently implement embeddings or a vector database in the provided code.

Part 1 - Project At A Glance

Topic	Project-specific explanation
Project name	ClaimSense AI, package name <code>claimsense-ai-personal</code> .
Problem statement	Insurance trainees need quick, source-backed answers from claim manuals, especially for rules like documentation deadlines, theft requirements, roof settlement rules, and workers' comp reporting.
Why solve it	Manuals are long, policy language is precise, and a generic chatbot may invent or generalize rules. This project tries to answer only from indexed manual passages and show citations.
Proposed solution	A React web app where a signed-in user asks a question. The system retrieves relevant manual chunks and returns an answer with cited excerpts.
Main objective	Help claim trainees ask natural-language questions and receive grounded, verifiable answers from insurance manuals.
Key features	Authentication screen, manual repository sidebar, PDF/DOCX upload, retrieval question box, citation cards, follow-up suggestions, local fallback retrieval, dark/light theme, tests for retrieval logic.
Technologies used	React, TypeScript, Vite, Tailwind CSS, shadcn/Radix UI components, Supabase Auth/Storage/Postgres/Edge Functions, Deno, Lovable AI gateway, Gemini model, Vitest.
Input	A trainee question such as: What documentation is required for a stolen-vehicle comprehensive claim? Optional input: uploaded PDF/DOCX manual.
Output	A grounded answer, citation list, page/section metadata, excerpts, and follow-up questions.

Role Of Each Technology

Technology	Role in this project
React	Builds the browser interface and updates the page when conversation state changes.
TypeScript	Adds type checking for objects like chat turns, citations, manual chunks, and Supabase table rows.
Vite	Runs the development server and builds the frontend into static files.
Tailwind CSS	Styles the app using utility classes.

Technology	Role in this project
Radix UI / shadcn-style components	Provides accessible UI primitives such as sheets, toasts, tooltips, buttons, and layout components.
Supabase Auth	Handles real user sessions when configured.
mockAuth	Provides development/demo login if Supabase is unavailable or not configured.
Supabase Storage	Stores uploaded manual files in the manuals bucket.
Supabase Postgres	Stores uploaded_manuals and manual_chunks.
Supabase Edge Functions	Runs ingest and retrieve backend logic close to Supabase.
Lovable AI gateway + Gemini	Chunks uploaded documents and generates final natural-language answers from retrieved passages.
Vitest	Tests tokenization, scoring, retrieval, and citation shape.

End-To-End Workflow

```

User signs in
-> selects or uploads a manual
-> asks a claim question
-> frontend validates query length
-> frontend calls Supabase retrieve function
-> retrieve loads seed + uploaded chunks
-> retrieve ranks chunks by token overlap
-> retrieve sends top chunks + question to Gemini through Lovable gateway
-> Gemini returns answer + used citation ids + followups
-> frontend displays answer and CitationCard components
-> if cloud call fails, browser localRetrieve creates a fallback answer

```

Project Explanations For Different Time Limits

Time	Answer to practice
5 minutes	ClaimSense AI is an insurance training assistant. The user signs in, chooses an indexed manual repository, and asks a claims question in plain English. The React frontend first tries to call a Supabase Edge Function named retrieve. That function combines a seed insurance corpus with uploaded manual chunks from PostgreSQL, scores passages by query-token overlap, selects the top passages, and sends only those passages plus the question to a Gemini model through the Lovable AI gateway. The model is instructed to answer only from the supplied passages and cite them as [#1], [#2], and so on. The frontend displays the answer, citation cards, and follow-up questions. The project also has an ingest Edge Function: uploaded PDF/DOCX files go to Supabase Storage, are parsed or sent to Gemini for chunk extraction, and their chunks are saved in manual_chunks. If Supabase or the function is unavailable, the app falls back to a local retrieval engine built from static chunks in src/data/manuals.ts.
2 minutes	ClaimSense AI helps insurance trainees get verified answers from claim manuals. It is a React + TypeScript frontend backed by Supabase. A question is sent to the retrieve function, relevant manual passages are ranked, and Gemini generates a citation-backed answer using only those passages. Uploaded manuals can be stored in Supabase Storage, chunked by the ingest function, and saved in Postgres. A local fallback lets the demo work without the cloud backend.
1 minute	ClaimSense AI is a RAG-style insurance manual assistant. Users ask policy questions, the app retrieves relevant manual chunks, and a Gemini model writes a grounded answer with citations. The frontend is React/TypeScript, the backend uses Supabase Edge Functions, Storage, and Postgres, and the project includes local fallback retrieval for demo reliability.
30 seconds	ClaimSense AI is a React and Supabase app that answers insurance claim questions from indexed manuals. It retrieves relevant passages, asks Gemini to answer only from those passages, and displays citations so trainees can verify every answer.

Part 2 - Explain The Project Like I Am A Beginner

Imagine a new claims trainee has a long stack of insurance manuals on a desk. The trainee asks: What documents do I need for a stolen vehicle claim? A normal chatbot might answer from memory. ClaimSense AI instead opens the relevant

manual sections, finds the clauses that mention stolen vehicles and documentation, and then writes an answer while showing exactly which clauses it used.

Running example used throughout this guide: A trainee asks, "What documentation is required to file a stolen-vehicle comprehensive claim?"

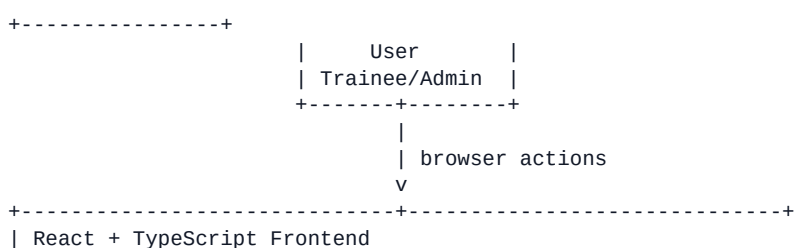
Beginner Workflow With One Running Example

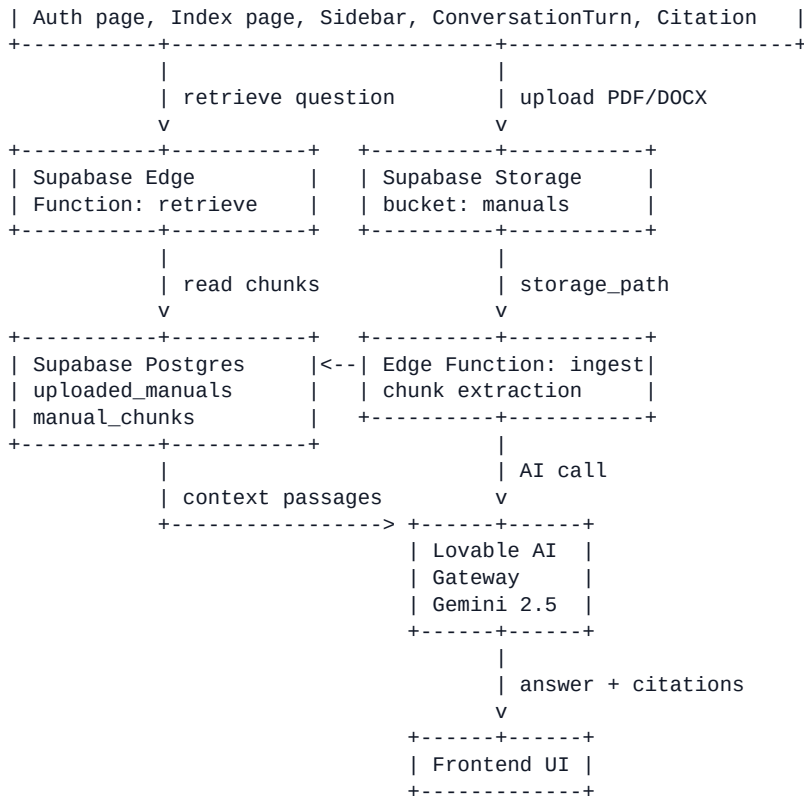
Stage	Input	Processing	Output
User input	Question typed into textarea.	Frontend trims whitespace and checks max length of 800 characters.	Clean query string.
Frontend state	Clean query.	Adds a ChatTurn with loading=true so the UI shows a skeleton answer.	Visible pending question.
Remote retrieval attempt	Body: { query: clean query }.	Calls supabase.functions.invoke('retrieve').	Either response data or remote error.
Backend retrieval	Query plus seed/uploaded chunks.	Tokenizes query, scores chunks, sorts by score, keeps top 4.	Relevant passages like auto-2, auto-1, auto-3.
AI synthesis	Top passages plus query.	Gemini is told to answer only from supplied passages and cite [#n].	Answer, used citation ids, followups.
Frontend display	Answer JSON.	Updates the matching turn, renders markdown bold and citation badges.	User sees answer and citation cards.
Fallback	If remote function fails.	localRetrieve uses static CHUNKS and rule-based synthesize().	Demo answer still appears.

Every Major Component

Component	What enters	What it does	What comes out	Next stop
Auth page	Email/password or Google click.	Creates a Supabase session or mock session in development.	Session object.	Index page.
Index page	Session, selected manual, question.	Manages conversation state and calls retrieval.	ChatTurn updates.	ConversationTurn component.
ArchiveSidebar	Manual list, uploaded records, file input.	Shows repositories, handles manual upload, updates uploaded list.	Selected manual id or uploaded manual record.	Index retrieval or ingest function.
retrieve Edge Function	HTTP POST JSON query.	Loads chunks, ranks them, calls Gemini.	Answer JSON.	Frontend.
ingest Edge Function	storage_path, filename, title.	Downloads file, extracts DOCX or sends PDF to model, saves chunks.	manual_id, chunk_count, status.	Sidebar refresh / database.
Postgres database	Uploaded manual metadata and chunks.	Persists searchable manual content.	Rows returned to retrieve/sidebar.	Edge functions/frontend.
Gemini through Lovable gateway	Question plus retrieved passages.	Generates grounded answer or extracts chunks.	Structured tool-call JSON.	Edge functions.

Part 3 - System Architecture





Explain Every Box And Arrow

Box/Arrow	Explanation
User -> Frontend	The trainee interacts with the browser: signs in, uploads manuals, selects a repository, asks a question.
Frontend	React components hold UI state and convert user actions into Supabase SDK calls.
Frontend -> retrieve	A POST-style Edge Function invocation sends the question in JSON.
retrieve -> Postgres	The function reads uploaded chunks whose parent manual status is ready.
retrieve -> Gemini	The function sends the top passages and query to the AI model through the Lovable gateway.
Gemini -> retrieve	Gemini returns structured JSON: answer, used_citation_ids, followups.
retrieve -> Frontend	The function returns answer and citation objects to the React app.
Frontend -> Storage	Uploaded PDF/DOCX files are saved in a Supabase Storage bucket named manuals.
Frontend -> ingest	After upload, frontend invokes ingest with file path and metadata.
ingest -> Postgres	The function creates uploaded_manuals and manual_chunks records.

Natural Spoken Architecture Answer

"The project is a React and TypeScript web app backed by Supabase. The frontend handles authentication, manual selection, uploads, and the chat interface. For answering a question, the frontend calls a Supabase Edge Function called retrieve. That function gathers built-in sample manual chunks plus any uploaded chunks from Postgres, ranks them against the query, sends the best passages to Gemini through the Lovable AI gateway, and returns an answer with citations. There is also an ingest function for PDF/DOCX uploads: it stores the file in Supabase Storage, extracts or asks Gemini to chunk the manual, and saves those chunks in Postgres. For development reliability, the frontend has mock authentication and local retrieval fallback, so the demo can still work when Supabase is not configured."

Part 4 - Complete End-To-End Execution

Trace: Stolen Vehicle Documentation Question

Step	Component	Input	Technical processing	Output
1	User	What documentation is required to file a stolen-vehicle comprehensive claim?	Types the question and presses Retrieve.	Form submit event.
2	Index.tsx	Form event + input state.	Prevents default browser form submission, trims text, rejects empty or >800 chars.	trimmed query.
3	Index.tsx	trimmed query.	Clears textarea, creates crypto.randomUUID(), appends loading ChatTurn.	turns includes pending answer.
4	Supabase client	{ query: trimmed }.	Calls supabase.functions.invoke('retrieve').	HTTP request to Edge Function.
5	retrieve function	Request JSON.	Handles OPTIONS, checks LOVABLE_API_KEY, validates query is a string.	Valid query or error response.
6	retrieve function	Seed chunks + uploaded chunks.	Tokenizes query and chunks. Counts overlap. Sorts descending. Selects top 4.	Top chunks, likely auto-2, auto-1, auto-3.
7	AI gateway	System prompt + user prompt + passages.	Gemini 2.5 Flash is asked to use only passages and return a tool-call object.	answer, used_citation_ids, followups.
8	retrieve function	AI response.	Parses tool call JSON and maps used citation ids to citation objects.	JSON response.
9	Index.tsx	JSON response.	Updates the matching ChatTurn: loading=false, answer/citations/followups set.	Renderable chat turn.
10	ConversationTurn	ChatTurn.	Splits bold syntax and [#n] citations into styled spans and superscript badges.	Grounded answer UI.
11	CitationCard	Citation objects.	Displays manual, section, page, heading, excerpt, copy button, expand/collapse.	Verifiable source cards.

Input -> Processing -> Output: Input: stolen vehicle claim question. Processing: tokenize, rank, send top passages to Gemini. Output: answer listing police report within 24 hours, title/registration, keys/remotes, SIU cooperation, with citations.

Part 5 - Technologies Used

Technology	What it is	How used	If removed	Alternatives
React	A JavaScript library for building UI from reusable components.	Used for pages/components such as Index, Auth, ArchiveSidebar, ConversationTurn.	Without it, the app would need another UI framework or manual DOM handling.	Vue, Angular, Svelte.
TypeScript	JavaScript with static types.	Defines Session, ChatTurn, ManualChunk, Supabase Database types.	More runtime mistakes could slip through, such as missing citation fields.	Plain JavaScript, Flow.
Vite	A fast frontend dev/build tool.	Runs dev server on port 8080 and builds dist/.	Need another bundler to transform TS/React and optimize assets.	Webpack, Parcel, Next.js.
Tailwind CSS	Utility-first CSS framework.	Most UI styling is className utilities in components.	Would need handwritten CSS or a component framework's styling system.	CSS Modules, styled-components, Bootstrap.
Radix UI/shadcn components	Accessible UI primitives and locally composed UI components.	Sheet, toast, tooltip, dialogs, buttons, etc.	More custom UI behavior and accessibility work.	Material UI, Chakra UI, Headless UI.

Technology	What it is	How used	If removed	Alternatives
Supabase	Backend-as-a-service built on Postgres.	Auth, Storage, Postgres, Edge Functions.	Would need custom backend, database, auth, storage hosting.	Firebase, custom Node/Express backend, Appwrite.
PostgreSQL	Relational database.	Stores uploaded_manuals and manual_chunks.	Uploaded documents would not persist beyond localStorage/demo.	MongoDB, MySQL, SQLite.
Supabase Edge Functions	Server-side functions deployed near Supabase, written for Deno.	retrieve and ingest backend logic.	Secrets and AI calls would have to happen in the browser, which is unsafe.	Vercel functions, AWS Lambda, Cloudflare Workers.
Deno	Runtime used by Supabase Edge Functions.	Runs TypeScript functions and imports remote/npm modules.	Need to rewrite functions for Node or another server runtime.	Node.js, Bun.
Lovable AI gateway	Hosted gateway endpoint for model calls.	Functions call https://ai.gateway.lovable.dev/v1/chat/completions.	Would call Google Gemini API directly or another provider.	Google AI Studio API, OpenAI API, Anthropic API.
Gemini 2.5 Flash	Fast LLM used in actual code.	Generates answers and extracts chunks through function/tool calls.	Only local deterministic responses would be available.	OpenAI GPT, Claude, Llama.
mammoth	DOCX raw-text extractor.	ingest extracts DOCX text before sending to the AI chunker.	DOCX uploads would be harder to parse.	docx4js, libreoffice conversion, custom XML parsing.
Vitest	Test runner for Vite projects.	Tests retrieval utilities and basic setup.	Less confidence in retrieval behavior.	Jest, Playwright component tests.
React Router	Client-side routing.	Routes /, /auth, and catch-all NotFound.	Would need manual location handling or another router.	TanStack Router, Next.js routing.
React Query	Server state library.	Provider is configured, but current source does not materially use queries/mutations.	Not much impact in current code; future data fetching could be less organized.	SWR, direct useEffect calls.

Part 6 - Core Concepts I Must Understand

Frontend And Backend

Item	Explanation
Simple definition	Frontend is the part users see; backend runs trusted work on a server.
Real-world analogy	Restaurant: frontend is the menu and waiter; backend is the kitchen.
Technical explanation	The frontend sends requests; the backend validates, reads data, calls models, and returns responses.
How it is used here	React is the frontend. Supabase Edge Functions are the backend.
Example	User asks about theft docs; frontend sends query; backend retrieves passages.
Interview question	Why split frontend and backend?
Strong interview answer	Because model secrets, database service keys, and heavy processing should stay on the backend, while the frontend focuses on interaction.

API

Item	Explanation
Simple definition	An API is a controlled way for one part of software to talk to another.

Item	Explanation
Real-world analogy	A service counter: you submit a form, the counter returns a result.
Technical explanation	The app invokes Edge Functions using JSON over HTTP-like calls.
How it is used here	supabase.functions.invoke('retrieve') and invoke('ingest') are API calls.
Example	Request body: { query: 'stolen vehicle documentation' }.
Interview question	What API does your app use?
Strong interview answer	The frontend uses Supabase SDK APIs for Auth, Storage, database reads, realtime, and Edge Function calls.

HTTP And JSON

Item	Explanation
Simple definition	HTTP moves requests/responses; JSON is a text format for structured data.
Real-world analogy	Courier package: HTTP is delivery, JSON is the labeled content inside.
Technical explanation	Edge Functions return JSON responses with answer/citations or errors.
How it is used here	retrieve validates req.json() and returns application/json.
Example	{ answer: '...', citations: [...], followups: [...] }.
Interview question	Why JSON?
Strong interview answer	JSON is simple, browser-native, and works well between React, Supabase, and Edge Functions.

Authentication

Item	Explanation
Simple definition	Authentication proves who the user is.
Real-world analogy	Showing an ID card before entering an office.
Technical explanation	Supabase Auth stores and refreshes sessions. mockAuth simulates a session in development.
How it is used here	Auth.tsx signs in users; Index.tsx redirects unauthenticated users to /auth.
Example	Email/password creates Supabase session; demo mode creates localStorage mock session.
Interview question	How is login handled?
Strong interview answer	Supabase Auth is the main path, with a development-only mock fallback so the app can be demonstrated without cloud config.

Authorization

Item	Explanation
Simple definition	Authorization decides what an authenticated user is allowed to do.
Real-world analogy	Having an ID card does not mean you can enter every room.
Technical explanation	RLS policies decide table/storage access. In this project, policies allow anyone to read/insert/update uploaded manuals and read/insert chunks.
How it is used here	The current project prioritizes demo accessibility over strict access control.
Example	Anyone can read uploaded_manuals according to the migration.
Interview question	Is authorization strong here?

Item	Explanation
Strong interview answer	Not yet. The provided RLS policies are permissive, so production should restrict records by user or organization.

Database And Tables

Item	Explanation
Simple definition	A database stores structured data; tables are organized lists of similar records.
Real-world analogy	Spreadsheet tabs with rules and relationships.
Technical explanation	Postgres tables <code>uploaded_manuals</code> and <code>manual_chunks</code> store uploaded document metadata and searchable excerpts.
How it is used here	<code>manual_chunks.manual_id</code> references <code>uploaded_manuals.id</code> .
Example	A manual row can have many chunk rows.
Interview question	What tables exist?
Strong interview answer	<code>uploaded_manuals</code> stores document metadata/status; <code>manual_chunks</code> stores extracted passages linked by <code>manual_id</code> .

RAG

Item	Explanation
Simple definition	Retrieval-Augmented Generation means retrieve facts first, then generate an answer from those facts.
Real-world analogy	Open the textbook before answering, instead of guessing from memory.
Technical explanation	A retriever selects relevant chunks; an LLM uses the selected chunks as context.
How it is used here	The <code>retrieve</code> function ranks chunks and sends the top passages to Gemini.
Example	Theft query -> auto-2 passage -> Gemini answer with <code>[#1]</code> .
Interview question	Is this a full RAG system?
Strong interview answer	It is RAG-style. The provided code has retrieval plus generation, but retrieval is token scoring rather than embeddings/vector search.

Tokenization

Item	Explanation
Simple definition	Tokenization splits text into searchable pieces.
Real-world analogy	Breaking a sentence into index cards.
Technical explanation	<code>tokenize_lowercases</code> , removes punctuation except hyphens, splits whitespace, removes stopwords and short tokens.
How it is used here	Used by <code>score()</code> in local retrieval.
Example	What documentation is required? -> documentation, required.
Interview question	Why tokenize?
Strong interview answer	So query words and chunk words can be compared consistently.

Stopwords

Item	Explanation
Simple definition	Stopwords are common words that usually do not help search.
Real-world analogy	Ignoring filler words when finding the main topic.
Technical explanation	The project filters words like the, and, what, is, for.
How it is used here	Improves simple ranking by focusing on meaningful terms.
Example	what is the coverage for roof -> coverage, roof.
Interview question	Why remove stopwords?
Strong interview answer	They appear everywhere and can make irrelevant passages look relevant.

Scoring And Ranking

Item	Explanation
Simple definition	Scoring gives each chunk a relevance number; ranking sorts by that number.
Real-world analogy	Sorting resumes by how many required skills match the job description.
Technical explanation	local score boosts heading/section matches more than body matches; Edge score uses simpler token overlap.
How it is used here	Chooses top 4 chunks for answer generation.
Example	Heading match for theft documentation beats generic text match.
Interview question	How do you find relevant passages?
Strong interview answer	The current implementation uses token overlap scoring with field boosts in the frontend fallback.

LLM

Item	Explanation
Simple definition	A large language model generates text from a prompt.
Real-world analogy	A very skilled writing assistant that needs source material to stay factual.
Technical explanation	Gemini receives instructions, manual passages, and the trainee query.
How it is used here	Used for chunk extraction in ingest and answer synthesis in retrieve.
Example	Prompt says use ONLY supplied passages.
Interview question	How do you reduce hallucination?
Strong interview answer	By limiting context to retrieved passages, instructing the model not to invent facts, and showing citations.

Prompt

Item	Explanation
Simple definition	A prompt is the instruction and input given to the model.
Real-world analogy	A work order given to an assistant.
Technical explanation	The retrieve function has a systemPrompt with strict rules and a userPrompt with passages and query.
How it is used here	Controls answer style, citation behavior, and grounding.
Example	Cite inline like [#1].
Interview question	What is in your prompt?
Strong interview answer	Rules for strict retrieval, concise trainee-friendly answer, and no invented forms or page numbers.

Citations

Item	Explanation
Simple definition	Citations show which source supports an answer.
Real-world analogy	Footnotes in a textbook.
Technical explanation	The model returns <code>used_citation_ids</code> ; the backend maps them to manual, section, page, heading, excerpt.
How it is used here	Displayed by CitationCard.
Example	[#1] Part E - Required Documentation for Theft Claims.
Interview question	Why citations?
Strong interview answer	Insurance rules need verification; citations let trainees check the exact source clause.

Local Fallback

Item	Explanation
Simple definition	Fallback is a backup path used when the preferred path fails.
Real-world analogy	If the elevator is down, use stairs.
Technical explanation	Index.tsx first tries retrieve; if unavailable, localRetrieve answers from static CHUNKS and localStorage chunks.
How it is used here	Makes demos work without Supabase/Gemini.
Example	Remote error -> console.warn -> localRetrieve.
Interview question	Why add fallback?
Strong interview answer	It improves reliability for development and interviews, but production should monitor backend failures rather than silently relying on simplified logic.

Environment Variables And Secrets

Item	Explanation
Simple definition	Environment variables configure software without hardcoding values.
Real-world analogy	Writing a locker combination on a private note, not on the public door.
Technical explanation	VITE_SUPABASE_URL and VITE_SUPABASE_PUBLISHABLE_KEY configure the frontend. LOVABLE_API_KEY and service role keys are server-side secrets.
How it is used here	Used by Supabase client and Edge Functions.
Example	.env.example documents required values.
Interview question	Where are secrets stored?
Strong interview answer	Frontend gets only public Supabase values; AI and service role keys should be stored as Supabase Edge Function secrets.

Testing

Item	Explanation
Simple definition	Testing checks expected behavior automatically.
Real-world analogy	A checklist that runs every time you change the system.

Item	Explanation
Technical explanation	Vitest runs unit tests for tokenize, score, and localRetrieve.
How it is used here	Confirms retrieval returns expected answers/citations for sample queries.
Example	19 tests passed after sandbox-approved run.
Interview question	What did you test?
Strong interview answer	I tested tokenization, stopwords removal, ranking behavior, retrieval results, no-match behavior, and citation shape.

Build And Deployment

Item	Explanation
Simple definition	Build converts source code into production files.
Real-world analogy	Packing raw ingredients into a ready-to-ship product.
Technical explanation	Vite builds dist/index.html, CSS, and JS assets.
How it is used here	Verified npm run build succeeds, with warnings about Browserslist and large chunk size.
Example	dist/assets/index-DN7ATLi9.js is about 583.52 kB minified.
Interview question	How would you deploy it?
Strong interview answer	Host the static frontend on Vercel/Netlify/Supabase hosting and deploy Edge Functions plus migrations to Supabase.

Part 7 - Code Walkthrough

Entry Point: src/main.tsx

main.tsx is the first frontend file that runs in the browser. It imports ReactDOM's createRoot, imports App, imports global CSS, finds the HTML element with id root, and renders the App component inside it.

```
createRoot(document.getElementById("root")!).render(<App />);
```

The exclamation mark tells TypeScript: I expect root to exist. In an interview, say this is safe because index.html contains the root element; however, a defensive production variant could check for null.

App.tsx

App.tsx wires the global providers and routes. ThemeProvider controls light/dark mode. QueryClientProvider is ready for React Query server-state usage. TooltipProvider, Toaster, and Sonner provide UI feedback. BrowserRouter defines three routes: /, /auth, and a catch-all NotFound page.

Index.tsx - The Main Chat Page

Code area	What it does	Interview explanation
State variables	session, authReady, turns, input, activeManual, submitting, sidebarOpen.	The page state tracks who is logged in, what the user typed, which manual is selected, and what chat turns should render.
Auth useEffect	Reads mock session first, then subscribes to Supabase auth changes and redirects unauthenticated users.	This protects the main page and keeps UI in sync with session changes.
Auto-scroll useEffect	Scrolls feed to bottom whenever turns changes.	Improves chat usability.
Textarea resize useEffect	Sets height based on scrollHeight up to 128px.	Keeps the input comfortable without letting it take over the screen.

Code area	What it does	Interview explanation
submitQuery	Validates query, creates loading turn, calls remote retrieve, falls back to localRetrieve, updates turn.	This is the central user action path.
clearConversation	Clears turns array and shows toast.	Session-level UI reset; not persisted to database.

Simplified submitQuery

```

async function submitQuery(query) {
  const trimmed = query.trim();
  if (!trimmed || submitting) return;
  if (trimmed.length > 800) showError();

  addLoadingTurn(trimmed);

  try {
    let data = await callSupabaseRetrieve(trimmed);
    if (!data) data = await localRetrieve(trimmed, activeManual);
    updateTurnWithAnswer(data);
  } catch (error) {
    updateTurnWithError(error.message);
  } finally {
    setSubmitting(false);
  }
}

```

retrieval.ts - Local Retrieval Logic

Function	Input	Processing	Output
tokenize	Any string.	Lowercase, remove punctuation except hyphens, split, remove stopwords, keep tokens length > 2.	Token list.
score	Query and ManualChunk.	Counts matches with boosts: heading/section x3, manual title x1.5, body x1, plus coverage bonus.	Numeric relevance score.
loadLocalChunks	None.	Reads claimsense_uploaded_chunks from localStorage.	ManualChunk array or empty array.
localRetrieve	query, activeManualId.	Combines seed + local chunks, optionally filters by manual, ranks, top 4, calls synthesize.	RetrievalResult.
synthesize	query, chunks.	Uses rule branches for known topics, otherwise stitches retrieved passages.	answer + followups.

retrieve Edge Function

The retrieve function is the production-style backend answer path. It validates the request, loads uploaded chunks from Postgres, ranks chunks, builds context, calls Gemini through Lovable's gateway, parses a tool-call response, maps citation ids to citation objects, and returns JSON.

```

const { query } = await req.json();
if (!query || typeof query !== "string") return 400;

const uploaded = await loadUploadedChunks();
const ALL = [...SEED, ...uploaded];
const top = ALL.map(c => ({ chunk: c, s: score(query, c) }))
  .filter(r => r.s > 0)
  .sort((a, b) => b.s - a.s)
  .slice(0, 4)
  .map(r => r.chunk);

```

ingest Edge Function

The ingest function supports manual uploads. It inserts a processing record, downloads the uploaded file, extracts DOCX text with mammoth or base64-encodes PDF bytes, asks Gemini to emit structured chunks, inserts those chunks, then marks the manual ready. If anything fails, it marks the manual failed with an error message.

ArchiveSidebar.tsx

ArchiveSidebar is more than navigation. It lists seed manuals, fetches uploaded manuals from Supabase, subscribes to realtime changes on uploaded_manuals, validates file type and size, uploads files, invokes ingest, and stores a local fallback manual/chunk if cloud ingestion fails.

ConversationTurn.tsx And CitationCard.tsx

ConversationTurn renders each question/answer pair. It shows a skeleton while loading, an error message on failure, or answer plus citations on success. CitationCard displays source metadata and supports copy plus expand/collapse.

Error Handling

- Frontend query validation blocks empty and over-800-character prompts.
- Remote retrieve failures are caught and localRetrieve is used as fallback.
- Retrieve returns 400 for missing query and 500 for missing secrets or unknown errors.
- Ingest marks the uploaded manual as failed if chunking or database insertion fails.
- Auth falls back to mock mode if Supabase fetch fails during sign-in.

Part 8 - Data Flow

```
Question text
-> trim and validate
-> ChatTurn loading state
-> retrieve Edge Function
-> seed chunks + uploaded manual_chunks
-> tokenization
-> relevance score
-> top 4 chunks
-> prompt context
-> Gemini response
-> parsed answer/citations/followups
-> React state update
-> ConversationTurn + CitationCard UI
```

Manual Upload Data Flow

```
PDF/DOCX file
-> frontend validates extension and 15 MB limit
-> Supabase Storage bucket manuals
-> ingest Edge Function receives storage_path
-> uploaded_manuals row status=processing
-> file download from storage
-> DOCX text extraction or PDF base64
-> Gemini chunk extraction
-> manual_chunks rows
-> uploaded_manuals status=ready, chunk_count=N
-> sidebar refresh/realtime update
```

Part 9 - Database

Database used: Supabase PostgreSQL. The project also uses Supabase Storage for raw manual files. The provided migrations create two application tables and one storage bucket.

```
uploaded_manuals (one row per uploaded document)
  id UUID primary key
  title text
```

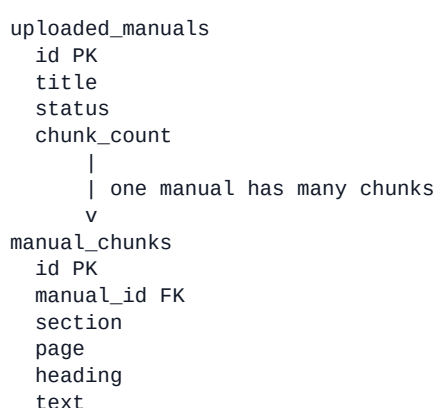
```

code text
edition text
source_filename text
storage_path text
status text
error text nullable
chunk_count integer
created_at timestampz

manual_chunks (many rows per uploaded manual)
id UUID primary key
manual_id UUID foreign key -> uploaded_manually.id
ordinal integer
section text
page integer
heading text
text text
created_at timestampz

```

Schema Diagram



Example Records

Table	Example record
uploaded_manually	id=uuid, title='Auto Comprehensive Upload', code='USR', edition='Uploaded', source_filename='auto.pdf', storage_path='1717000000-auto.pdf', status='ready', chunk_count=12.
manual_chunks	id=uuid, manual_id=<same manual id>, ordinal=0, section='Part E', page=27, heading='Required Documentation for Theft Claims', text='If your covered auto is stolen...'

Common Mistake: Do not claim the project uses a vector database table. The provided migrations do not create embeddings or vector columns.

Part 10 - AI / Machine Learning Explanation

The AI/ML part solves two tasks: first, chunking uploaded manuals during ingest; second, generating final answers during retrieval. The code uses an LLM, not a trained custom machine learning model. There is no training code in the project.

Topic	Project fact
Model problem	Turn retrieved manual passages into a trainee-friendly answer; extract useful chunks from uploaded manuals.
Input to model	For retrieve: top passages + query. For ingest: PDF bytes as base64 or DOCX raw text + chunking instructions.
Preprocessing	DOCX is extracted with mammoth; PDF is base64-encoded; query retrieval uses tokenization before model call.
Model/algorithm	Actual code calls google/gemini-2.5-flash via Lovable AI gateway.
Training	Not present in the provided code/documentation.

Topic	Project fact
Inference	The deployed function sends prompts and gets model output at request time.
Output	Structured JSON through function/tool calls: answer, used_citation_ids, followups or chunks.
Evaluation	No AI quality evaluation suite is present. Vitest covers local retrieval behavior, not model answer quality.
Limitations	Model may still produce imperfect answers; grounding depends on retrieved chunks; no embeddings; uploaded PDF extraction relies on model support.

Prompt, Tokens, Context, Hallucination, Grounding

A prompt is the instruction sent to the model. Tokens are pieces of text the model processes. Context is the set of manual passages included with the question. Hallucination means the model invents a plausible but unsupported answer. Grounding means the answer is tied to supplied source passages.

In this project, grounding is attempted by putting only the top passages into the prompt and instructing the model: use ONLY these passages, cite [#n], and never invent forms, sections, or page numbers.

Embeddings, Vector Databases, RAG

Embeddings are numeric representations of text meaning. A vector database stores those numbers for semantic search. RAG combines retrieval and generation. The documentation describes semantic search, but the actual code does not create embeddings or use a vector database. The actual retrieval is keyword/token overlap.

Part 11 - API Explanation

retrieve API

```
Endpoint: Supabase Edge Function named retrieve
Method: POST-style function invocation
Headers: handled by Supabase SDK; function supports CORS
Body:
{
  "query": "What documentation is required to file a stolen-vehicle comprehensive claim?"
}

Response:
{
  "answer": "To file a stolen-vehicle comprehensive claim...",
  "citations": [
    {
      "id": "auto-2",
      "manual": "Auto Comprehensive",
      "section": "Part E - Duties After an Accident or Loss",
      "page": 27,
      "heading": "Required Documentation for Theft Claims",
      "excerpt": "If your covered auto is stolen..."
    }
  ],
  "followups": ["When exactly does rental reimbursement coverage end after a theft?"]
}
```

ingest API

```
Endpoint: Supabase Edge Function named ingest
Method: POST-style function invocation
Body:
{
  "storage_path": "1717000000-auto.pdf",
  "filename": "auto.pdf",
  "title": "Auto Comprehensive"
}

Response:
```

```
{
  "manual_id": "<uuid>",
  "chunk_count": 12,
  "status": "ready"
}
```

GET/PUT/PATCH/DELETE are not directly exposed as custom REST endpoints in this project. Supabase SDK internally uses HTTP methods for database/storage operations, but the project code mainly uses insert, update, select, upload, download, signIn, signUp, and Edge Function invocation.

Part 12 - Why This Technology / Design?

Decision	Why chosen	Alternative	Why not alternative	Tradeoff
React + TypeScript	Chosen for component-based UI and safer types.	Plain JavaScript or Vue.	React ecosystem fits Vite, shadcn-style components, Supabase examples, and interview-friendly frontend structure.	TypeScript adds learning overhead.
Supabase	Chosen to avoid building auth, storage, database, realtime, and functions from scratch.	Custom Express backend + Postgres.	Faster for a project; Postgres remains production-grade.	RLS must be configured carefully; vendor dependency.
Edge Functions	Keep AI secrets and service-role database access off the browser.	Call Gemini directly from frontend.	Frontend secrets would be exposed; backend functions are safer.	Deno runtime differs from typical Node.
RAG-style design	Insurance answers need source grounding.	Plain chatbot.	A plain chatbot may hallucinate or miss company manuals.	Quality depends on retrieval accuracy.
Keyword scoring now	Simple, testable, works for small corpus and demo.	Embeddings/vector search.	Embeddings would be stronger semantically but require more infrastructure.	Keyword scoring misses synonyms and deeper meaning.
Local fallback	Demo remains usable when Supabase is unavailable.	Hard fail if backend down.	Better developer experience and interview demo reliability.	Fallback behavior may differ from production.
Citation cards	Make answers verifiable.	Show only answer text.	Interviewers like traceability in high-stakes domains.	More UI complexity.
Vitest	Fast tests that align with Vite.	Jest.	Vitest config is simpler in Vite projects.	Less universal than Jest in older codebases.

Part 13 - Challenges And Solutions

Problem	Why it happens	How solved	Why that solution	Alternative
AI hallucination	LLMs can invent plausible insurance rules.	Prompt says use only supplied passages and cite sources.	Grounding reduces unsupported answers.	Add automated citation verification.
Wrong passage retrieval	Keyword overlap can miss synonyms or rank generic text highly.	Field boosts and top-4 ranking.	Simple and testable for small corpus.	Use embeddings and hybrid search.
Backend unavailable	Supabase/Gateway may fail or be unconfigured.	localRetrieve fallback.	Keeps demo usable.	Show degraded-mode banner and retry queue.
Secrets exposure	AI keys cannot be shipped to browser.	Store LOVABLE_API_KEY in Edge Function secrets.	Backend-only access is safer.	Use direct provider secret management in another cloud.

Problem	Why it happens	How solved	Why that solution	Alternative
PDF/DOCX complexity	Documents may have tables, scans, headers, odd formatting.	DOCX uses mammoth; PDF sent to model as base64.	Works for many simple manuals.	Use OCR and robust document parsing pipeline.
Permissive RLS	Current policies allow broad access.	Demo policies prioritize usability.	Easy setup for training/demo.	Add user/org ownership policies.
Large frontend bundle	Build reports >500 kB chunk.	Current single bundle still builds successfully.	Fine for prototype.	Code split routes and lazy-load UI libraries.
Duplicate seed corpus	Seed chunks exist in frontend and retrieve function.	Kept in sync manually.	Simple fallback and backend seed behavior.	Move seed corpus to shared data store.
No model evaluation	Generated answers are hard to validate automatically.	Unit tests cover local retrieval only.	Tests stable deterministic logic.	Add golden answer tests and human evals.
Upload failure after DB insert	Ingest may fail after creating uploaded_manuals row.	Catch errors and mark status failed with message.	User/admin can see failed state.	Use transactional workflow where possible.

Hardest Part - Interview Answer

Hypothetical but realistic answer: The hardest part was balancing AI flexibility with source accuracy. In insurance, a confident wrong answer is dangerous. I solved this by making retrieval happen before generation, passing only selected passages to the model, requiring citations, and displaying citation cards so the user can verify the source.

Bug And Debugging - Interview Answer

Hypothetical but realistic answer: One bug I would discuss is retrieval ranking. A generic chunk can match many common words and beat a more relevant section. I debugged this by writing tests around example queries, then boosting heading and section matches so a directly named topic ranks higher than a body-only match.

Part 14 - Limitations

- No true semantic embeddings/vector search in the provided code.
- Permissive database/storage policies are not production-safe.
- Edge Functions have `verify_jwt=false` in `supabase/config.toml`.
- Seed corpus is duplicated between frontend and backend.
- Local fallback is rule-based and may not behave like the remote model.
- No persistent chat history table is present.
- No user-to-manual ownership model is present.
- PDF chunking depends on AI model capability; scanned PDFs may fail or be inaccurate.
- No automated AI answer evaluation is present.
- Build shows a large chunk warning and stale Browserslist warning.

If I Had More Time

I would add proper user/org authorization, replace keyword-only retrieval with hybrid search using embeddings plus keyword matching, move the seed corpus into the database, add persistent conversation history, add automated evaluation for answer faithfulness, and split the frontend bundle for performance.

Part 15 - Scalability

Scale	What happens	Likely bottlenecks	Improvements
10 users	Current design should handle demo usage easily.	Mostly none except API secrets/config.	Keep logs and monitor errors.
1,000 users	Edge Functions and Supabase can handle moderate traffic, but AI gateway cost/latency matters.	LLM latency, repeated same queries, database scans.	Add caching, indexes, rate limits, query logs.
100,000 users	Need production architecture work.	AI cost, concurrency, database load, upload processing, security boundaries.	Queues for ingest, worker pipeline, org-based sharding, CDN, caching, observability.
1 million users	Requires serious distributed design.	Global latency, tenant isolation, vector index size, cost controls, abuse prevention.	Load balancing, multi-region deployment, managed vector DB/hybrid search, autoscaling workers, billing/rate policy.

Part 16 - Security

Area	How it applies	Interview-safe improvement
Authentication	Supabase Auth and Google OAuth path exist; mockAuth exists only in development.	Disable/mockAuth guard in production and require real sessions.
Authorization	RLS policies currently allow anyone to read/insert/update certain tables.	Add user_id/org_id columns and restrict each tenant's data.
Input validation	Query max length 800; uploads allow only PDF/DOCX and max 15 MB in frontend and PDF path.	Also validate MIME/content server-side.
SQL injection	Supabase query builder reduces manual SQL injection risk.	Still validate data and avoid raw SQL for user input.
API security	verify_jwt=false for ingest/retrieve.	Require JWT for production functions unless intentionally public.
Secrets	LOVABLE_API_KEY and service role keys are backend secrets.	Never put them in VITE_ variables or frontend code.
Data privacy	Uploaded manuals may be proprietary.	Use private bucket, strict RLS, encryption, audit logs.
HTTPS	Supabase and hosted frontend should use HTTPS.	Required for secure auth and clipboard/browser APIs.
Access control	Manual ownership is not present.	Add ownership model before production.

Part 17 - Testing And Debugging

Verified checks: npm test passed with 2 test files and 19 tests. npm run build passed. The build reported a stale Browserslist data warning and a chunk-size warning for a 583.52 kB minified JS file.

Test Case	Input	Expected Output	Possible Failure
Tokenize simple query	Roof Settlement	roof, settlement	Case or punctuation not normalized.
Stopwords	what is the coverage for the roof	coverage, roof only important tokens	Common words inflate scores.
Roof ranking	roof depreciation windstorm hail	pc-2 ranks highest	Wrong policy chunk selected.
Auto theft retrieval	stolen vehicle documentation theft claim	Answer mentions police report and auto-2 citation	Missing theft duties.
Workers comp retrieval	first report injury deadline	Answer includes 7 days and wc-1	Wrong manual chosen.

Test Case	Input	Expected Output	Possible Failure
No match	deep sea fishing regulations	No matching passages message	Fabricated answer.
Upload invalid file	notes.txt	Only PDF/DOCX error	Unsupported file accepted.
Oversize upload	>15 MB file	File exceeds 15 MB error	Large upload overloads ingest.
Missing secret	retrieve without LOVABLE_API_KEY	500 missing secret error	Unclear backend failure.
Auth redirect	No session	Navigate to /auth	Protected page visible.

Debugging Approach

- Reproduce with a concrete query from the starter prompts.
- Check browser console for remote Edge Function fallback warnings.
- Inspect network/function response JSON.
- Use tests to isolate tokenize/score/localRetrieve behavior.
- Check Supabase table status for uploaded_manuals: processing, ready, failed.
- Check Edge Function logs for missing secrets, AI gateway errors, and chunk parsing errors.

Part 18 - Deployment

```
Developer machine
-> npm install
-> configure .env for Supabase public values
-> run tests and build
-> commit to Git/GitHub
-> create Supabase project
-> apply migrations
-> create/set Edge Function secrets
-> deploy ingest and retrieve functions
-> deploy frontend dist to Vercel/Netlify/Supabase hosting
-> users access production URL over HTTPS
```

Docker is not present in the provided project. A possible future deployment could use Docker for a custom backend, but this project as provided is better described as a static Vite frontend plus Supabase backend services.

Part 19 - Interview Question Bank

Beginner

Question: What problem does ClaimSense AI solve?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: ClaimSense AI solves the problem of finding precise insurance claim rules inside long manuals. It retrieves relevant passages and generates an answer with citations instead of relying on unsupported model memory.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: Who is the target user?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The primary user is an insurance claims trainee or adjuster who needs to quickly understand eligibility rules, documentation duties, deadlines, and exceptions from manuals.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is the input and output?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The main input is a natural-language claim question, and the output is an answer, citation objects, source excerpts, page/section metadata, and follow-up questions.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is a citation in this app?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A citation is a link between a statement in the answer and a retrieved manual passage. In the UI it appears as [#n] and is expanded in CitationCard with manual, section, page, heading, and excerpt.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is a manual chunk?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A manual chunk is a small self-contained passage from a manual. The app searches chunks instead of whole documents because smaller passages are easier to rank, cite, and fit into an LLM context.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: Why does the app need login?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Login is handled by Supabase Auth when configured. In development, mockAuth can create a localStorage demo session so the app remains usable without cloud setup.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is React used for?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: React is used to build the interactive browser UI from components such as Auth, Index, ArchiveSidebar, ConversationTurn, and CitationCard.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is Supabase used for?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Supabase provides Auth, Storage, Postgres, realtime subscriptions, and Edge Functions. It lets the project have a backend without writing a separate always-on server.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What happens when the user asks a question?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is the local fallback?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The app first tries the remote retrieve function. If that fails, Index.tsx catches the error and calls localRetrieve, which searches static chunks plus localStorage chunks and returns a deterministic answer.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is the main page of the app?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is the Auth page?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: Why is TypeScript useful?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is Vite?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What does the sidebar show?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What does the Retrieve button do?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: What happens if the top chunk is wrong?

How to answer: The answer may be grounded in the wrong source, so I would improve ranking, add hybrid search, and evaluate retrieval precision.

Question: What is a starter prompt?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The retrieve prompt contains strict instructions: answer only from supplied manual passages, cite with [#n], be concise, and never invent forms, sections, or page numbers.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is a follow-up question?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is localStorage used for?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What tests are present?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Vitest tests tokenization, stopword removal, ranking, localRetrieve outputs, no-match behavior, manual filtering fallback, and citation shape. In my run, 19 tests passed.

Possible follow-up: What is not tested yet?

How to answer: Remote Edge Functions, Supabase integration, upload processing, auth flows, UI behavior, and LLM answer faithfulness need additional tests.

Intermediate

Question: Explain the submitQuery flow.

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How does localRetrieve rank chunks?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A manual chunk is a small self-contained passage from a manual. The app searches chunks instead of whole documents because smaller passages are easier to rank, cite, and fit into an LLM context.

Possible follow-up: What happens if the top chunk is wrong?

How to answer: The answer may be grounded in the wrong source, so I would improve ranking, add hybrid search, and evaluate retrieval precision.

Question: Why are heading matches boosted?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Heading and section terms are boosted because they usually name the actual topic. This prevents a generic body-text match from beating a passage whose heading directly says theft documentation or roof settlement.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How does the retrieve function build context?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The retrieve Edge Function validates the query, loads seed and uploaded chunks, ranks passages, builds model context, calls Gemini 2.5 Flash through the Lovable gateway, parses structured output, and returns answer/citations/followups.

Possible follow-up: What happens if the top chunk is wrong?

How to answer: The answer may be grounded in the wrong source, so I would improve ranking, add hybrid search, and evaluate retrieval precision.

Question: How does the app handle failed remote retrieval?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How are uploaded manuals shown in the sidebar?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in ArchiveSidebar. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores manual_chunks, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: How does the ingest function process DOCX?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in ArchiveSidebar. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores manual_chunks, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: How does the ingest function process PDF?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in ArchiveSidebar. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores manual_chunks, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: What database relationship exists?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The database has uploaded_manuals for document metadata/status and manual_chunks for extracted passages. manual_chunks.manual_id is a foreign key to uploaded_manuals.id, so one manual can have many chunks.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is RLS and how is it configured here?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The provided RLS policies are permissive, so production should add user_id or organization_id columns, private storage, and policies that restrict each user to their own manuals.

Possible follow-up: What is the first production security fix?

How to answer: Require authenticated Edge Functions and change RLS so manuals/chunks are scoped to a user or organization.

Question: What is the role of Supabase Storage?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Supabase provides Auth, Storage, Postgres, realtime subscriptions, and Edge Functions. It lets the project have a backend without writing a separate always-on server.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How are citations rendered?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A citation is a link between a statement in the answer and a retrieved manual passage. In the UI it appears as [#n] and is expanded in CitationCard with manual, section, page, heading, and excerpt.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What does renderAnswer do?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How is authentication state tracked?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Login is handled by Supabase Auth when configured. In development, mockAuth can create a localStorage demo session so the app remains usable without cloud setup.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: Why use Edge Functions for AI calls?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is the difference between seed chunks and uploaded chunks?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A manual chunk is a small self-contained passage from a manual. The app searches chunks instead of whole documents because smaller passages are easier to rank, cite, and fit into an LLM context.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: What does the no-match path return?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What build warnings appeared?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The production build passed. It warned that Browserslist data is stale and that one JavaScript chunk is larger than 500 kB, so code splitting would be a future performance improvement.

Possible follow-up: What is not tested yet?

How to answer: Remote Edge Functions, Supabase integration, upload processing, auth flows, UI behavior, and LLM answer faithfulness need additional tests.

Question: How would you debug a wrong citation?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A citation is a link between a statement in the answer and a retrieved manual passage. In the UI it appears as [#n] and is expanded in CitationCard with manual, section, page, heading, and excerpt.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is the role of Vitest?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Vitest tests tokenization, stopword removal, ranking, localRetrieve outputs, no-match behavior, manual filtering fallback, and citation shape. In my run, 19 tests passed.

Possible follow-up: What is not tested yet?

How to answer: Remote Edge Functions, Supabase integration, upload processing, auth flows, UI behavior, and LLM answer faithfulness need additional tests.

Question: Why is query length limited?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How does realtime update sidebar data?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is the purpose of types.ts?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How does error handling work in ingest?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in ArchiveSidebar. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores manual_chunks, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuscripts.status to failed, stores the error message, and returns an error response.

Question: What does `verify_jwt=false` mean?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The main security improvements are authenticated Edge Functions, private storage, strict RLS, server-side MIME validation, audit logs, rate limits, and keeping AI/service keys only in backend secrets.

Possible follow-up: What is the first production security fix?

How to answer: Require authenticated Edge Functions and change RLS so manuals/chunks are scoped to a user or organization.

Advanced

Question: Is this true semantic search?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The documentation describes semantic search and other Gemini versions, but the actual code uses Gemini 2.5 Flash and token-overlap retrieval. I would be honest and call embeddings/vector search a future improvement, not a current feature.

Possible follow-up: How exactly would you add embeddings?

How to answer: Create embeddings for each chunk during ingest, store them in pgvector or a vector DB, embed the query at retrieval time, and combine semantic similarity with keyword scoring.

Question: How would you add embeddings?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The documentation describes semantic search and other Gemini versions, but the actual code uses Gemini 2.5 Flash and token-overlap retrieval. I would be honest and call embeddings/vector search a future improvement, not a current feature.

Possible follow-up: How exactly would you add embeddings?

How to answer: Create embeddings for each chunk during ingest, store them in pgvector or a vector DB, embed the query at retrieval time, and combine semantic similarity with keyword scoring.

Question: How would you prevent hallucination more strongly?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The project reduces hallucination by retrieving passages first, prompting the model to use only those passages, and showing citations. A stronger production version should verify citations and reject unsupported claims.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you secure tenant data?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The provided RLS policies are permissive, so production should add `user_id` or `organization_id` columns, private storage, and policies that restrict each user to their own manuals.

Possible follow-up: What is the first production security fix?

How to answer: Require authenticated Edge Functions and change RLS so manuals/chunks are scoped to a user or organization.

Question: How would you scale ingest?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in `ArchiveSidebar`. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores `manual_chunks`, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates `uploaded_manuals.status` to failed, stores the error message, and returns an error response.

Question: How would you evaluate answer faithfulness?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you handle scanned PDFs?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in ArchiveSidebar. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores manual_chunks, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: How would you reduce AI cost?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you design caching?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you handle model timeouts?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The actual code calls google/gemini-2.5-flash through the Lovable AI gateway. It uses the model for answer synthesis in retrieve and chunk extraction in ingest.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you version manuals?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you avoid duplicated seed data?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you improve chunking quality?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A manual chunk is a small self-contained passage from a manual. The app searches chunks instead of whole documents because smaller passages are easier to rank, cite, and fit into an LLM context.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you improve prompt safety?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The retrieve prompt contains strict instructions: answer only from supplied manual passages, cite with [#n], be concise, and never invent forms, sections, or page numbers.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you make retrieval hybrid?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you add observability?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you prevent prompt injection in manuals?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The retrieve prompt contains strict instructions: answer only from supplied manual passages, cite with [#n], be concise, and never invent forms, sections, or page numbers.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you add audit trails?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you handle organization roles?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you code split the app?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you harden Edge Functions?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you test Edge Functions?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Vitest tests tokenization, stopword removal, ranking, localRetrieve outputs, no-match behavior, manual filtering fallback, and citation shape. In my run, 19 tests passed.

Possible follow-up: What is not tested yet?

How to answer: Remote Edge Functions, Supabase integration, upload processing, auth flows, UI behavior, and LLM answer faithfulness need additional tests.

Question: How would you handle concurrent uploads?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in ArchiveSidebar. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores manual_chunks, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: How would you migrate from demo to production?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How would you measure retrieval precision?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Project-specific

Question: What is in `src/data/manuals.ts`?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is in `src/utls/retrieval.ts`?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is `STARTER_PROMPTS`?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The retrieve prompt contains strict instructions: answer only from supplied manual passages, cite with `[#n]`, be concise, and never invent forms, sections, or page numbers.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What does `MAX_CHARS` do?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What does activeManual control?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is the role of ArchiveSidebar?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What happens in onFile?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What tables are created by the migration?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The database has uploaded_manuals for document metadata/status and manual_chunks for extracted passages. manual_chunks.manual_id is a foreign key to uploaded_manuals.id, so one manual can have many chunks.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What does manual_chunks_manual_id_idx optimize?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A manual chunk is a small self-contained passage from a manual. The app searches chunks instead of whole documents because smaller passages are easier to rank, cite, and fit into an LLM context.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is the seed corpus?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What model is actually used in code?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The actual code calls google/gemini-2.5-flash through the Lovable AI gateway. It uses the model for answer synthesis in retrieve and chunk extraction in ingest.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What model does documentation mention?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The actual code calls google/gemini-2.5-flash through the Lovable AI gateway. It uses the model for answer synthesis in retrieve and chunk extraction in ingest.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is the contradiction in the docs?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The documentation describes semantic search and other Gemini versions, but the actual code uses Gemini 2.5 Flash and token-overlap retrieval. I would be honest and call embeddings/vector search a future improvement, not a current feature.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What does synthesize do?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What are the known answer branches?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What happens for an unrelated query?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What does mockAuth do?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is the Lovable integration?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: How does Google sign-in happen?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What does CitationCard copy?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A citation is a link between a statement in the answer and a retrieved manual passage. In the UI it appears as [#n] and is expanded in CitationCard with manual, section, page, heading, and excerpt.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What does uploaded_manuals.status mean?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in ArchiveSidebar. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores manual_chunks, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: How does the function load uploaded chunks?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A manual chunk is a small self-contained passage from a manual. The app searches chunks instead of whole documents because smaller passages are easier to rank, cite, and fit into an LLM context.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: Why does retrieve need a service role key?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: What happens if the top chunk is wrong?

How to answer: The answer may be grounded in the wrong source, so I would improve ranking, add hybrid search, and evaluate retrieval precision.

Question: What is the PDF size limit?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in ArchiveSidebar. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores manual_chunks, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: What does mammoth do?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What does the tool-call schema enforce?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What is a ChatTurn?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What did npm test verify?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Vitest tests tokenization, stopword removal, ranking, localRetrieve outputs, no-match behavior, manual filtering fallback, and citation shape. In my run, 19 tests passed.

Possible follow-up: What is not tested yet?

How to answer: Remote Edge Functions, Supabase integration, upload processing, auth flows, UI behavior, and LLM answer faithfulness need additional tests.

Question: What did npm build produce?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The production build passed. It warned that Browserslist data is stale and that one JavaScript chunk is larger than 500 kB, so code splitting would be a future performance improvement.

Possible follow-up: What is not tested yet?

How to answer: Remote Edge Functions, Supabase integration, upload processing, auth flows, UI behavior, and LLM answer faithfulness need additional tests.

Question: What is not present in the project?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Not present in the provided code: embeddings, vector database, Docker, persistent chat history, strict tenant authorization, custom model training, and automated LLM faithfulness evaluation.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Scenario-based

Question: The retrieve function returns 500. What do you check?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The retrieve Edge Function validates the query, loads seed and uploaded chunks, ranks passages, builds model context, calls Gemini 2.5 Flash through the Lovable gateway, parses structured output, and returns answer/citations/followups.

Possible follow-up: What happens if the top chunk is wrong?

How to answer: The answer may be grounded in the wrong source, so I would improve ranking, add hybrid search, and evaluate retrieval precision.

Question: The answer has no citations. What do you check?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A citation is a link between a statement in the answer and a retrieved manual passage. In the UI it appears as [#n] and is expanded in CitationCard with manual, section, page, heading, and excerpt.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: A PDF upload fails. What do you check?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in ArchiveSidebar. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores manual_chunks, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: A user sees another user's manuals. What is wrong?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The provided RLS policies are permissive, so production should add user_id or organization_id columns, private storage, and policies that restrict each user to their own manuals.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: Search misses a synonym. What improvement would you make?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: My first improvements would be stricter security, hybrid retrieval with embeddings, persistent conversation history, automated answer evaluation, better document parsing, and bundle code splitting.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: The model invents a page number. How do you respond?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The actual code calls google/gemini-2.5-flash through the Lovable AI gateway. It uses the model for answer synthesis in retrieve and chunk extraction in ingest.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: The app works locally but not production. What do you check?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: Build chunk is too large. What do you do?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A manual chunk is a small self-contained passage from a manual. The app searches chunks instead of whole documents because smaller passages are easier to rank, cite, and fit into an LLM context.

Possible follow-up: What is not tested yet?

How to answer: Remote Edge Functions, Supabase integration, upload processing, auth flows, UI behavior, and LLM answer faithfulness need additional tests.

Question: Supabase is not configured. What happens?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Supabase provides Auth, Storage, Postgres, realtime subscriptions, and Edge Functions. It lets the project have a backend without writing a separate always-on server.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: Local fallback gives different answer than cloud. How explain?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The app first tries the remote retrieve function. If that fails, Index.tsx catches the error and calls localRetrieve, which searches static chunks plus localStorage chunks and returns a deterministic answer.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: An uploaded manual stays processing forever. What do you check?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in ArchiveSidebar. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores manual_chunks, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: A DOCX has no extractable text. What happens?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in ArchiveSidebar. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores manual_chunks, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: Gateway rate limits requests. What do you do?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: User uploads confidential documents. What must change?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in ArchiveSidebar. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores manual_chunks, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: Question is over 800 characters. What happens?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: Realtime updates do not appear. What do you check?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: Citation copy button fails. Is app broken?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A citation is a link between a statement in the answer and a retrieved manual passage. In the UI it appears as [#n] and is expanded in CitationCard with manual, section, page, heading, and excerpt.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: The user asks an out-of-scope question. What should happen?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: A new manual type is needed. How add it?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: A strict interviewer asks why no vector DB. What say?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The documentation describes semantic search and other Gemini versions, but the actual code uses Gemini 2.5 Flash and token-overlap retrieval. I would be honest and call embeddings/vector search a future improvement, not a current feature.

Possible follow-up: How exactly would you add embeddings?

How to answer: Create embeddings for each chunk during ingest, store them in pgvector or a vector DB, embed the query at retrieval time, and combine semantic similarity with keyword scoring.

Follow-up / Cross-questioning

Question: Why not use ChatGPT directly?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A normal chatbot answers from general model knowledge, which may be outdated or unsupported. ClaimSense retrieves company/manual-specific passages first, then generates a grounded answer with citations.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: Why not Java?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Java could work for the backend, but this project is frontend-heavy and Supabase Edge Functions run TypeScript/Deno naturally. React + TypeScript keeps the main stack consistent.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: Why not Firebase?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Firebase is a valid alternative, but Supabase gives a relational Postgres database, SQL migrations, storage, auth, realtime, and functions, which fit structured manual/chunk metadata well.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: Why Supabase instead of custom backend?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Supabase provides Auth, Storage, Postgres, realtime subscriptions, and Edge Functions. It lets the project have a backend without writing a separate always-on server.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What if Gemini is wrong?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The actual code calls google/gemini-2.5-flash through the Lovable AI gateway. It uses the model for answer synthesis in retrieve and chunk extraction in ingest.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What if retrieval is wrong?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What if policies change?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What if a manual is huge?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What if many users upload files?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in ArchiveSidebar. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores manual_chunks, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: What if someone uploads malicious content?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Upload starts in ArchiveSidebar. The file is validated, uploaded to Supabase Storage, and sent to the ingest function. ingest creates a processing record, extracts chunks using mammoth or Gemini, stores manual_chunks, and marks the manual ready.

Possible follow-up: What happens if chunking fails?

How to answer: The ingest function catches the error, updates uploaded_manuals.status to failed, stores the error message, and returns an error response.

Question: What if the API key leaks?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The main security improvements are authenticated Edge Functions, private storage, strict RLS, server-side MIME validation, audit logs, rate limits, and keeping AI/service keys only in backend secrets.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What if database goes down?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: The database has uploaded_manuals for document metadata/status and manual_chunks for extracted passages. manual_chunks.manual_id is a foreign key to uploaded_manuals.id, so one manual can have many chunks.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What if user has no internet?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What if citations do not match answer?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: A citation is a link between a statement in the answer and a retrieved manual passage. In the UI it appears as [#n] and is expanded in CitationCard with manual, section, page, heading, and excerpt.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What if a claim rule is state-specific?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What if the interviewer asks for architecture?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What if asked about your contribution?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: Do not invent this. State your real contribution. If your role was study/maintenance, say you analyzed the architecture, traced the data flow, verified tests/build, and can explain limitations and improvements.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What if asked about limitations?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What if asked about future work?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: My first improvements would be stricter security, hybrid retrieval with embeddings, persistent conversation history, automated answer evaluation, better document parsing, and bundle code splitting.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Question: What if asked to explain to non-technical person?

What interviewer is testing: Whether you understand the project beyond memorized buzzwords.

Strong answer: I would answer by tracing the exact project flow: React UI receives the action, Supabase handles backend/database/storage work, retrieval selects passages, Gemini generates grounded text, and the UI displays citations. I would also clearly state any missing production features.

Possible follow-up: Can you connect that answer to the actual code?

How to answer: Name the specific file, function, or table, then separate what exists now from what you would improve later.

Part 20 - Project Cross-Questioning Simulation

Interviewer: Tell me about your project.

Candidate: One of my projects is ClaimSense AI, an insurance training assistant that answers claim-manual questions with citations. A trainee asks a question, the app retrieves relevant manual passages, and Gemini generates an answer using only those passages.

Interviewer: Why not just use a normal chatbot?

Candidate: Because insurance policy answers must be tied to exact clauses. A normal chatbot can give plausible but unsupported answers. Here, the value is retrieval and citations.

Interviewer: Is it using embeddings?

Candidate: The architecture document describes semantic search, but the provided code currently uses token-based scoring. I would call it RAG-style today and say embeddings are a planned production improvement.

Interviewer: Walk me through one request.

Candidate: The user submits a query in `Index.tsx`. The frontend validates it, adds a loading chat turn, invokes the `retrieve` Edge Function, receives `answer/citations/followups`, and renders them through `ConversationTurn` and `CitationCard`.

Interviewer: What happens inside `retrieve`?

Candidate: It validates the query, loads seed and uploaded chunks, scores chunks by token overlap, keeps top four, builds a prompt, calls Gemini 2.5 Flash through the Lovable gateway, parses the tool-call JSON, and returns answer JSON.

Interviewer: What if the backend is down?

Candidate: The frontend catches the remote failure and falls back to `localRetrieve`, which uses static manual chunks and deterministic synthesis branches.

Interviewer: How do uploads work?

Candidate: `ArchiveSidebar` validates PDF/DOCX and size, uploads to Supabase Storage, then invokes `ingest`. `ingest` stores a processing record, downloads the file, extracts chunks using `mammoth` for DOCX or model-based PDF processing, inserts `manual_chunks`, and marks the manual ready.

Interviewer: What tables are used?

Candidate: `uploaded_manuals` for document metadata and status, and `manual_chunks` for extracted passages. `manual_chunks.manual_id` is a foreign key to `uploaded_manuals.id`.

Interviewer: What is weak in the current security?

Candidate: RLS policies are permissive and the functions have `verify_jwt=false`. That is acceptable for a demo but not production. I would add user/org ownership, private storage, and authenticated functions.

Interviewer: What did you test?

Candidate: Vitest covers tokenization, stopword filtering, ranking examples, `localRetrieve` answers, no-match behavior, active manual behavior, and citation shape. The suite passed 19 tests.

Interviewer: What would you improve first?

Candidate: I would first harden security and then improve retrieval using hybrid search with embeddings plus keyword matching, because both accuracy and data privacy matter in insurance.

Part 21 - HR + Project Questions

Tell me about your project.

ClaimSense AI is a web app for insurance trainees. It lets them ask natural-language questions about claim manuals and returns source-backed answers with citations.

Why did you choose this project?

I chose it because insurance claim handling is detail-heavy, and mistakes can be costly. It is a good use case for retrieval-grounded AI rather than a generic chatbot.

What was your contribution?

A safe answer is: I studied and can explain the frontend flow, retrieval logic, Supabase backend, database schema, and testing approach. If you personally implemented parts, name only those parts truthfully.

What did you personally implement?

Not specified in the project files. Prepare your real contribution. Example if true: I implemented the retrieval utility, citation UI, and Supabase function integration.

What was the biggest challenge?

Keeping answers grounded. The project reduces risk by retrieving passages first and requiring citations.

What did you learn?

I learned how a React frontend, Supabase backend, and LLM prompting can work together in a RAG-style application.

What would you improve?

I would add secure authorization, embeddings/hybrid retrieval, persistent chat history, and answer quality evaluation.

What happens if the system fails?

Remote failure falls back to local retrieval in the frontend. In production I would also add monitoring, retries, and visible degraded-mode messaging.

Why should we believe you understand it?

I can trace a request from textarea to Edge Function to database/model and back to the citation UI, and I can also explain current limitations honestly.

Explain it to a non-technical person.

It is like a smart assistant that reads the company claim manuals before answering, then shows the exact manual section it used so the trainee can verify it.

Part 22 - 2-Minute Project Presentation

One of my projects is ClaimSense AI, an insurance training assistant for answering claim-manual questions with citations. The problem is that insurance manuals are long and precise, and a generic chatbot can easily give a confident but unsupported answer. ClaimSense solves this by using a RAG-style workflow: first retrieve relevant manual passages, then generate an answer only from those passages.

The frontend is built with React, TypeScript, Vite, Tailwind CSS, and reusable UI components. The backend uses Supabase for authentication, storage, PostgreSQL, realtime updates, and Edge Functions. When a trainee asks a question, the React app sends it to the retrieve Edge Function. The function ranks manual chunks, sends the top passages to Gemini through the Lovable AI gateway, and returns an answer with citations. Uploaded manuals go through the ingest function, which stores the file, extracts chunks, and saves them in the database.

The most important technical decision was grounding the answer in citations, because claim handling requires accuracy and traceability. The biggest limitation is that the current code uses token-based retrieval rather than true embeddings, so a production version should add hybrid search, stricter authorization, and automated faithfulness evaluation.

Part 23 - Rapid Revision Sheet

- Objective: answer insurance claim-manual questions with source citations.
- Frontend: React + TypeScript + Vite + Tailwind.
- Backend: Supabase Auth, Storage, Postgres, Edge Functions.
- AI: actual code uses Gemini 2.5 Flash through Lovable AI gateway.
- Main pages: Auth.tsx and Index.tsx.
- Main components: ArchiveSidebar, ConversationTurn, CitationCard, ErrorBoundary.
- Main utilities: mockAuth and localRetrieve.
- Database: uploaded_manuals and manual_chunks.
- Retrieval: tokenization + scoring + top 4 chunks.

- RAG status: RAG-style; no embeddings/vector DB in provided code.
- Upload: PDF/DOCX -> Storage -> ingest -> chunks -> Postgres.
- Answer: query -> retrieve -> Gemini -> citations -> UI.
- Security gap: permissive RLS and verify_jwt=false.
- Testing: 19 Vitest tests passed.
- Build: passed, with stale Browserslist and large chunk warnings.
- Top improvements: secure RLS, embeddings, persistent chat, evals, code splitting.

20 Most Likely Questions

- Tell me about ClaimSense AI.
- Why is RAG useful here?
- Is this true semantic search?
- Walk through one request.
- What happens if Supabase fails?
- How are citations generated?
- What is a manual chunk?
- How are uploaded manuals processed?
- What database tables are used?
- How does authentication work?
- What are the security limitations?
- Why use Supabase?
- Why use TypeScript?
- What did you test?
- What would you improve?
- How would you scale it?
- How would you add embeddings?
- How would you prevent hallucination?
- What build warnings exist?
- What is not present in the project?

Part 24 - Flashcards

Q1: What is ClaimSense AI?

A: A web app that answers insurance claim-manual questions with citations.

Project connection: It is the main project.

Q2: What is RAG?

A: Retrieval-Augmented Generation: retrieve sources first, then generate an answer.

Project connection: The project retrieves manual chunks before Gemini answers.

Q3: What is React?

A: A library for building user interfaces.

Project connection: Used for Auth, Index, sidebar, and answer UI.

Q4: What is TypeScript?

A: JavaScript with types.

Project connection: Used to define manual chunks, citations, chat turns, and database rows.

Q5: What is Vite?

A: A fast frontend build tool.

Project connection: Runs and builds the React app.

Q6: What is Supabase?

A: A backend platform with database, auth, storage, and functions.

Project connection: Provides most backend services.

Q7: What is Postgres?

A: A relational database.

Project connection: Stores uploaded manuals and manual chunks.

Q8: What is Supabase Storage?

A: File storage service.

Project connection: Stores uploaded PDF/DOCX manuals.

Q9: What is an Edge Function?

A: Server-side function deployed on Supabase.

Project connection: retrieve and ingest run there.

Q10: What is Gemini?

A: An LLM from Google.

Project connection: Used through Lovable gateway for generation and chunking.

Q11: What is tokenization?

A: Splitting text into useful words.

Project connection: Used by retrieval.ts scoring.

Q12: What are stopwords?

A: Common words removed from search.

Project connection: Words like the, and, what are filtered.

Q13: What is a chunk?

A: A smaller passage of a document.

Project connection: Manuals are searched as chunks.

Q14: What is a citation?

A: A reference to source material.

Project connection: CitationCard shows section, page, heading, excerpt.

Q15: What is localRetrieve?

A: Browser fallback retrieval function.

Project connection: Answers from static and localStorage chunks.

Q16: What is synthesize?

A: Function that builds local fallback answers.

Project connection: Has branches for roof, theft, power, WC, etc.

Q17: What is mockAuth?

A: Development-only fake session helper.

Project connection: Allows demo login without Supabase.

Q18: What is RLS?

A: Row Level Security.

Project connection: Configured permissively in the migration.

Q19: What is `verify_jwt=false`?

A: Function does not require a valid JWT.

Project connection: Current config uses it for ingest and retrieve.

Q20: What is mammoth?

A: DOCX text extraction library.

Project connection: Used in ingest for DOCX files.

Q21: What does `Index.tsx` do?

A: It is the main chat page.

Project connection: It manages session checks, input, turns, retrieval calls, and display state.

Q22: What does `Auth.tsx` do?

A: It handles sign-in and sign-up UI.

Project connection: It uses Supabase Auth, Lovable OAuth, and mockAuth fallback.

Q23: What does `ArchiveSidebar` do?

A: It shows manuals and handles uploads.

Project connection: It validates files, uploads to Storage, invokes ingest, and listens for realtime updates.

Q24: What does `ConversationTurn` do?

A: It renders one question-answer pair.

Project connection: It shows loading skeleton, error, answer, and citations.

Q25: What does `CitationCard` do?

A: It displays a source excerpt.

Project connection: It shows manual, section, page, heading, and copy/expand controls.

Q26: What is `uploaded_manuals`?

A: A database table for uploaded document metadata.

Project connection: It tracks title, file path, status, error, and chunk count.

Q27: What is `manual_chunks`?

A: A database table for extracted passages.

Project connection: retrieve reads ready chunks from this table.

Q28: What is `SUPABASE_SERVICE_ROLE_KEY`?

A: A powerful backend database key.

Project connection: Edge Functions use it to read/write data; it must never be exposed to the browser.

Q29: What is `LOVABLE_API_KEY`?

A: A backend secret for AI gateway access.

Project connection: retrieve and ingest need it to call the model.

Q30: What does `verify_jwt=false` mean?

A: The function does not require a signed-in user's JWT.

Project connection: It is convenient for demos but should be changed for production.

Q31: What is the 800 character limit?

A: A frontend query-size guard.

Project connection: Index.tsx rejects prompts longer than MAX_CHARS.

Q32: What is the 15 MB limit?

A: An upload-size guard.

Project connection: ArchiveSidebar blocks files over 15 MB; ingest also checks PDF bytes.

Q33: What is a no-match response?

A: A safe answer when no relevant passages are found.

Project connection: localRetrieve and retrieve avoid fabricating unsupported answers.

Q34: What is field boosting?

A: Giving more weight to important fields.

Project connection: local score gives heading/section matches more value than body matches.

Q35: What is a seed corpus?

A: Built-in sample manual data.

Project connection: src/data/manuals.ts and retrieve/index.ts both include seed chunks.

Q36: What is the duplication risk?

A: Same seed data exists in frontend and backend.

Project connection: They can drift unless moved to a shared database/source.

Q37: What is Realtime used for?

A: Live updates from database changes.

Project connection: ArchiveSidebar subscribes to uploaded_manuals changes.

Q38: What is the main documentation mismatch?

A: Docs mention semantic search and different Gemini versions.

Project connection: Code uses token scoring and google/gemini-2.5-flash.

Q39: What is not trained?

A: There is no custom ML model training.

Project connection: The project uses an external LLM at inference time.

Q40: What passed verification?

A: Tests and build.

Project connection: npm test passed 19 tests; npm run build succeeded with warnings.

Q41: What is the large chunk warning?

A: A build performance warning.

Project connection: The JS bundle is over 500 kB after minification.

Q42: What is the stale Browserslist warning?

A: Browser compatibility data is outdated.

Project connection: Run update tooling later to refresh caniuse-lite.

Q43: What is a production security priority?

A: Restrict access to user/org data.

Project connection: Add ownership columns, private storage, strict RLS, and JWT verification.

Q44: What is a production accuracy priority?

A: Improve retrieval quality.

Project connection: Add embeddings or hybrid search plus citation verification.

Q45: What is a production reliability priority?

A: Make failures observable.

Project connection: Add logging, retries, degraded-mode UI, and monitoring.

Q46: What is persistent chat history?

A: Saving conversations to a database.

Project connection: Not present in the provided project.

Q47: What is Docker?

A: A way to package apps in containers.

Project connection: Not present; deployment is more naturally static frontend + Supabase backend.

Q48: What is an LLM tool call?

A: A structured response requested from the model.

Project connection: retrieve asks Gemini to call respond with answer, used_citation_ids, followups.

Q49: What is chunk extraction?

A: Turning documents into smaller passages.

Project connection: ingest asks Gemini to emit chunks with section, page, heading, and text.

Q50: What is the safest interview habit?

A: Separate facts from future improvements.

Project connection: It keeps your explanation credible.

Part 25 - Final Can I Explain This Project? Checklist

- ☐ Can I explain the problem in simple words?
- ☐ Can I explain why insurance manuals need citations?
- ☐ Can I explain the architecture from user to frontend to backend to database/model and back?
- ☐ Can I trace the stolen-vehicle question end to end?
- ☐ Can I explain Auth.tsx and Index.tsx?
- ☐ Can I explain submitQuery?
- ☐ Can I explain localRetrieve, tokenize, score, and synthesize?
- ☐ Can I explain retrieve and ingest Edge Functions?
- ☐ Can I explain the database schema?
- ☐ Can I explain what is actually implemented versus documented?
- ☐ Can I say clearly that embeddings/vector DB are not present?
- ☐ Can I explain Supabase Auth, Storage, Postgres, and Edge Functions?
- ☐ Can I explain why secrets belong on the backend?
- ☐ Can I explain citations and CitationCard?
- ☐ Can I explain the local fallback?
- ☐ Can I explain testing results?
- ☐ Can I explain security gaps honestly?
- ☐ Can I explain scalability bottlenecks?
- ☐ Can I answer why each technology was chosen?
- ☐ Can I answer what I would improve next?
- ☐ Can I explain the project in 5 minutes, 2 minutes, 1 minute, and 30 seconds?